

Semana 9 — Control nodes e HUD

Semana 9

Control nodes e HUD

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade III — Resolver Problemas (Semanas 8–11)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender Control nodes como sistema universal de interface em tempo real, na mesma Scene Tree de qualquer outro Node do projeto
- Construir um Control simples vinculado a uma variável de gameplay já existente, entendendo o binding de dados entre lógica de jogo e interface
- Fundamentar CanvasLayer como camada de organização do HUD sobre a cena de jogo, e montar um HUD com múltiplos elementos
- Propor e implementar, com autonomia própria, quais dados de gameplay já existentes (vida, itens, progresso) compõem o HUD do grupo

Semana 9 — Control nodes e HUD

ENCONTRO 1

Control Nodes: Interface em Tempo Real

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 8 (animação contextual conectada a evento real) (15 min)
- Introdução: o problema de comunicar estado de jogo em tempo real; Control nodes na Scene Tree (20 min)
- Demonstração: containers e anchors como resolução de layout e ancoragem (20 min)
- Demonstração: construção guiada de um Control simples vinculado à vida do `HealthComponent` (35 min)
- Laboratório: cada grupo replica o Control simples sobre o próprio `HealthComponent` (30 min)
- Feedback e fechamento (15 min)



Todo o estado do jogo já existe — vida, itens, progresso. Por que o jogador ainda não vê nada disso na tela?

Pense no que falta entre o `HealthComponent` e o que aparece na tela.

O Problema: Estado Invisível ao Jogador

- Vida (`HealthComponent`), itens (`SaveComponent` / `ItemData`) e progresso (`GameManager` / `SaveData`) já existem e funcionam desde os Módulos 2 e 3
- Nada na tela comunica esse estado em tempo real ao jogador
- A resposta do Godot é o Control node: um Node como qualquer outro na Scene Tree, especializado em desenho e interação 2D de interface

Control Node: UI na Mesma Scene Tree do Jogo

- Control é um Node comum, na mesma árvore de tudo o mais — não um sistema externo separado da cena de jogo
- HUD, menus e telas de gameplay compartilham a mesma árvore, a mesma composição e os mesmos Signals já ensinados desde a Semana 5
- Três problemas recorrentes de qualquer UI: organização automática (containers), ancoragem relativa ao viewport (anchors) e binding de dados

Containers e Anchors: Organização e Ancoragem

- Containers (`HBoxContainer` / `VBoxContainer`) organizam múltiplos elementos automaticamente, sem posicionamento manual
- Anchors ancoram um elemento a uma posição relativa ao viewport, independente de resolução — não é posição fixa em pixels
- Os dois resolvem problemas diferentes: um organiza o conjunto, o outro ancora a posição

Sistema de UI no Godot × Unity

Godot

- Control: Node comum, na mesma Scene Tree de tudo o mais
- Ancoragem e organização de layout nativas e unificadas (anchors e containers built-in)

Unity

- Dois caminhos paralelos: uGUI (GameObject + RectTransform + Canvas) e UI Toolkit (documentos UXML/USS)
- Divisão entre dois sistemas de UI que a Unity ainda mantém

Binding de Dados: Ler, Nunca Duplicar

- Um Control não deve conter lógica de gameplay — apenas ler ou ser notificado sobre um dado que já existe em outro sistema
- `HealthComponent`, `SaveComponent`, `GameManager` continuam sendo os donos do estado; o Control apenas espelha esse estado
- Mesma separação de responsabilidades já cobrada desde o Code Review da Semana 7

Demonstração — Control Simples Vinculado à Vida

O que será construído:

- Um Control simples (Label) organizado por container e posicionado por anchor
- Binding do Label à vida atual do `HealthComponent` do Player

Por quê: dar a cada grupo a base direta para o HUD completo do Encontro 2, sem duplicar o estado de vida já existente.

Imagem sugerida

Objetivo: mostrar o Control node na mesma Scene Tree do jogo, lendo (não duplicando) o estado do HealthComponent.

Enquadramento: uma única árvore de cena do Player.

Elementos presentes: "Player" com filhos "HealthComponent" e "HUD" (Control), com uma seta pontilhada de "HealthComponent" para o Label dentro do "HUD" indicando leitura, não posse do dado.

Destaque visual: a seta pontilhada de leitura, contrastando com uma alternativa errada (variável de vida duplicada dentro do Control) marcada com um X.

Legenda sugerida: "O Control lê a vida do HealthComponent — nunca duplica o estado."

Arquitetura — Control como Nova Camada de Leitura

- O Control simples não altera `HealthComponent`, State Machine ou animação contextual da Semana 8
- Nenhum sistema de gameplay novo é criado nesta semana — apenas uma camada de exposição sobre o que já existe
- A conexão é sempre unidirecional: o sistema de gameplay expõe, o Control lê

Laboratório — Control Simples Vinculado à Vida

Cada grupo replica, no próprio projeto:

1. Um Control simples (Label) organizado por container
2. Ancoragem do Control a uma posição relativa ao viewport
3. Binding do Label à vida atual do próprio `HealthComponent`, sem duplicar o estado

Fechamento — Encontro 1

- Control simples funcional, exibindo em tempo real a vida atual do Player a partir do `HealthComponent`
- Layout organizado por container e posicionado por anchor
- HealthComponent, State Machine e animação contextual da Semana 8 sem nenhuma alteração
- Próximo passo: CanvasLayer e o desafio do HUD completo, no Encontro 2

Semana 9 — Control nodes e HUD

ENCONTRO 2

CanvasLayer e o Desafio do HUD Completo

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (Control simples vinculado à vida) (15 min)
- Demonstração: fundamentação de CanvasLayer e montagem guiada de um HUD com múltiplos elementos (25 min)
- Apresentação do desafio: cada grupo define quais dados compõem seu HUD e propõe a própria solução visual/de binding (15 min)
- Laboratório/Desafio: cada grupo monta seu HUD sobre CanvasLayer, conectando os dados escolhidos (50 min)
- Feedback formal: cada grupo apresenta e justifica as escolhas do próprio HUD (30 min)

Semana 9 — Control nodes e HUD



Um único Label de vida resolve o HUD completo do jogo?

Pense no que acontece quando vida, itens e progresso precisam aparecer juntos, sempre visíveis, independente da câmera 3D.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema: Compor Múltiplos Elementos Sempre Visíveis

- O Encontro 1 resolveu a interface em tempo real no nível de um único elemento
- Falta compor múltiplos elementos (vida, itens, progresso) em um HUD coeso
- O HUD precisa permanecer sempre visível, independente da câmera 3D ou da profundidade da cena

CanvasLayer: Camada 2D Independente do Mundo 3D

- CanvasLayer desenha seu conteúdo em uma camada 2D separada da renderização 3D do mundo
- Garante que o HUD nunca seja ocultado por geometria do cenário
- Organiza múltiplos Control nodes (vida, itens, progresso) sob um único CanvasLayer

Camada de HUD no Godot × Unity

Godot

- CanvasLayer: um único Node na mesma Scene Tree de tudo o mais

Unity

- uGUI: Canvas em modo Screen Space - Overlay
- UI Toolkit: `UIDocument` associado a um `PanelSettings` de overlay

Demonstração — HUD com Múltiplos Elementos

O que será construído:

- CanvasLayer com múltiplos Control nodes (vida, itens, progresso) organizados em um único HUD
- Reutilização exata dos dados já existentes no projeto, sem criar nenhum sistema de estado novo

Por quê: fixar a montagem guiada antes de abrir o espaço de decisão do desafio do encontro.

Imagem sugerida

Objetivo: mostrar o HUD completo organizado sobre um único CanvasLayer.

Enquadramento: árvore de cena com destaque de camada.

Elementos presentes: "CanvasLayer" como pai de três Controls — "Label Vida", "Label Itens", "Label Progresso" — sobrepostos a uma miniatura da cena 3D do jogo abaixo.

Destaque visual: a camada do CanvasLayer flutuando visivelmente acima da geometria 3D.

Legenda sugerida: "CanvasLayer: uma camada 2D sempre visível, independente da profundidade do mundo 3D."

Arquitetura — Do Dado Existente ao HUD

- Cada elemento do HUD é sempre um espelho de um dado que já existe em outro sistema: vida (`HealthComponent`), itens/progresso (`SaveComponent` / `SaveData` / `GameManager`)
- Nenhum novo sistema de estado deve ser criado exclusivamente para a interface
- O desafio é de decisão — quais dados, e que solução visual/de binding representa cada um



Desafio — HUD Próprio sobre CanvasLayer

Defina quais dados de gameplay já existentes (vida, itens, progresso) devem compor o HUD do seu grupo, propondo a própria solução visual e de binding para cada um, organizados sobre um único CanvasLayer.

OBJETIVOS

Entrega: Feedback formal — HUD funcional exibindo em tempo real ao menos vida e um segundo dado de gameplay já existente, com a escolha justificada perante o grupo e o professor.

Boas Práticas — HUD e Binding de Dados

- HUD sempre como espelho de um dado que já existe — nunca uma variável nova criada só para a interface
- Justificar a escolha de cada dado exibido pela relevância para o jogador, não por preferência arbitrária
- Um único CanvasLayer organizando todos os Controls do HUD, evitando camadas redundantes
- Testar cada Control isoladamente antes de compor o HUD completo

Fechamento — Encontro 2

- HUD funcional sobre CanvasLayer, exibindo em tempo real vida e ao menos um segundo dado de gameplay
- Escolha dos dados e solução visual justificada perante o grupo e o professor
- Nenhuma lógica de gameplay duplicada dentro dos Control nodes
- Segundo desafio da Unidade III sob Challenge Based Learning concluído

Resultado Esperado da Semana

- HUD funcional montado sobre CanvasLayer, com Control nodes exibindo em tempo real vida (`HealthComponent`) e um segundo dado já existente (itens ou progresso)
- Solução visual e de binding própria de cada grupo, apresentada e justificada em feedback formal
- Turma domina Control nodes, containers, anchors e CanvasLayer como sistema universal de interface em tempo real
- Comparação consolidada com uGUI/UI Toolkit da Unity

Checklist da Semana

- [] Control simples (Label) vinculado à vida do `HealthComponent`, organizado por container e posicionado por anchor
- [] CanvasLayer organizando múltiplos Control nodes em um único HUD
- [] HUD exibindo em tempo real vida e ao menos um segundo dado (`SaveComponent` / `SaveData` ou `GameManager`)
- [] Nenhuma lógica de gameplay duplicada dentro dos Control nodes
- [] Escolha dos dados do HUD justificada em feedback formal

Próximos Passos — Semana 10

O HUD construído nesta semana ganha um novo consumidor na Semana 10: o

`InventoryComponent`, que reutiliza os itens já modelados como `ItemData` desde a Semana 6 e passa a expor sua própria `InventoryUI` sobre os mesmos princípios de Control node e binding de dados fundamentados aqui. A Semana 10 também retoma e amplia diretamente o contrato `Interactable` da Semana 5, conectando-o ao novo sistema de inventário.

Leitura recomendada: Godot Docs — User Interface (UI); Unity Manual (consulta comparativa) — UI Toolkit, Canvas.